

Programmed sums to infinity?

GFDL-1.3-or-later

A reader wrote in with two points about the programmed sums. First, that a sum whose terms get smaller, such as $1/i^2$, should be added from the last term to the first, so the small ones are not lost in a large running total. Second, a statement or maybe question that stopping when the total stops changing is not safe. On most calculators the first point is right.

On the C47/R47, for most series's the sequence changes nothing, as long as Σ_n does the adding. It matters as soon as your own program keeps the running total.

Section 3 answers the second point. A new function came out of it.

REV	DATE	DESCRIPTION
0.1	2026-08-02	JM. First issue.

1 Why the direction makes no difference

A register holds 34 digits. The running total inside Σ_n holds 75. Your program is handed the counter, works out one term and leaves it in **X**. The engine takes it from there and adds it to that 75 digit total (spoiler, this is the secret). Your program never sees the total. Only the finished answer comes back to a register, rounded to the 34 digits a register holds.

Forty one spare digits sit under the answer while the sum runs, so no term can be lost in the total, however small it has become (realistically) and in whatever order the terms arrive. Adding the small ones first would protect digits that were never at risk, the 41 that you never see.

Using $i\Sigma_n$ loses no digits either. Long integers are exact at any size, up to the 1000 digit limit.

The direction of counting is determined by the start, end and step parameters: Σ_n starts at **Z**, steps by **X** and stops at **Y**, so counting down is a larger start, a smaller finish and a negative step. The engine reorders nothing.

2 When it does matter

The other way to do a sum is to manually add the term to a variable yourself, in a loop or within the term program, with RCL, + and STO. People do that to watch the total build, to stop on a condition, or to use the interim values. The running total is then an ordinary register of 34 digits, or shorter if SDIGS has been reduced, and the 75 digit total of Section 1 is not involved.

The adding internally, at 34 digits, against externally, at 75, is illustrated in Table 1. The Σ_n examples agree with each other and with the rounded true value to every digit they show. The manual + lines do not agree with each other: counting up gives 1.644332, wrong in the fourth decimal, counting down gives 1.644834, right.

Table 1 Sum of $1/n^2$ for $n = 1$ to 10000. The four sums ran at SDIGS 7; the true value is given in full.

	HOW THE SUM WAS MADE	RESULT
a.	True value	1.644834071848059769806081833310311
b.	Σ_n , counting up	1.644834
c.	Σ_n , counting down	1.644834
d.	+ into a variable, up	1.644332
e.	+ into a variable, down	1.644834

2.1 Why the upward + line is wrong

Consider one term's addition in the + up count, line (d) above. The counter has reached $n = 2000$, so the term is $1/2000^2$, and the running total has grown to about 1.644834.

Table 2 The term at $n = 2000$ added to the running total of the upward sum, at SDIGS 7.

	VALUE	
running total	1.644834	seven digits, so the last one is worth 0.000001
the term at 2000	0.00000025	smaller than that last digit
+ gives back	1.644834	the term is gone

Every term from $n = 1414$ onwards is smaller than the last digit of that total, so the upward sum spends its final eight and a half thousand iterations adding nothing. Those lost terms are worth 0.0005 between them, which is the gap between the fourth line and the true value.

2.2 Why counting down saves it

Now the same term in line (e) above. Counting down, $n = 2000$ comes near the end of the run, and the running total holds only the terms from 10000 down to 2001, so it is still small.

Table 3 The same term at $n = 2000$ added to the running total of the downward sum, at SDIGS 7.

	VALUE	
running total	0.0003998800	seven digits, so the last one is worth 0.0000000001
the term at 2000	0.00000025	two thousand times larger than that last digit
+ gives back	0.0004001300	the term is in, with room to spare

Seven digits are counted from the first digit that is not zero, not from the decimal point. A small total carries its seven digits further to the right, so it can still take a small term. The large terms arrive last, when the total has grown to meet them, and by then every small one is already inside it. That is the letter's first point, and on this route it holds.

2.3 Why the Σ_n lines were never at risk

Lines (b) and (c) are correct to all seven digits they show, and SDIGS 7 is rounding just as hard there. It rounds in different places. Each term is rounded to seven digits on its way out of your program, and the finished answer is rounded to seven on its way back to **X**. The additions in between are not rounded at all.

Put numbers on it. Say your program hands the engine $f(n) = 9.8 \times 10^{35}$ on one iteration and $f(n+1) = 1.2 \times 10^{-35}$ on the next. Written out, the total is

[illegible]

which runs 72 digits from that first **9** to that last **2**. The engine holds 75, so both terms are in the total, whole and unrounded. A register could not have taken it: at 34 digits the small term falls off the end and is gone.

That is where the limit sits, and you can find it before writing the program. Take the largest total the run will reach, and the smallest term it will ever add. If $\log_{10}$ of the one divided by the other comes out above 75, terms are being lost inside the engine, and you should do the adding yourself in rising order of size: the smallest $f(n)$ first, which for a series that dies away means starting at the largest n .

The 75 digits give you no more digits in the answer, since the answer is a register and SDIGS has the last word on it. What they do is stop the ten thousand additions in between from costing you anything.

3 Stopping early, safely

Adding terms until the total stops changing looks like a safe rule but is not always safe. The terms that no longer move the total are still worth something between them.

Section 2 has already shown it happening. In the upward + sum at SDIGS 7 the total stopped changing at $n = 1414$, and the eight and a half thousand terms after that were worth 0.0005 together, five hundred times the last digit that had still been moving. A rule watching the total would have stopped there and been wrong in the fourth decimal.

Σ_n carries no such rule. It counts from **Z** to **Y** in steps of **X** and stops when the count is done, and it stays that way.

Such a function in C47/R47 did not exist and was created for this interesting typical case. $\infty\Sigma_n$ takes the same three numbers and the same program as Σ_n , and counts the same way, but stops when it deems further change to be unachievable. The test is short, because the run is finite. Write m for the iteration just done, N for the last iteration the run would make, $f(m)$ for the term that iteration added, and $\Sigma(m)$ for the total after it. The run stops when both of these hold:

$$|f(m)| < |f(m-1)| \quad (1)$$

$$|f(m)| \times (N - m) < \frac{|\Sigma(m)|}{10^{\text{SDIGS}}} \quad (2)$$

with two arbitrary limits defined in firmware:

Ignore initial settling : $m \geq 10$ (3)

Only when diminishing : $m \geq 50$ (4)

Formula 1 has to hold at every iteration, not merely at the one in hand, and it is judged only from the iteration given by Formula 3. From the tenth iteration on, any increase in the term disqualifies the early stop. Formula 2 monitors the possible rise. A term of exactly zero is skipped instead: it says nothing about the terms after it, so a function with a root at a whole number keeps counting.

Essentially, nothing left in the run is deemed capable of changing the current sum, so the programmed sum can stop at m . $\infty \sum_n$ leaves the number of iterations it ran in $\mathbf{Y}$, so a run that stopped early and one that

counted to the end can be told apart. A complex sum always counts to the end and **Y** says so: the test needs the size of a term, and the size of a complex number is not free.

Formula 4 forces a measure of certainty that the run's terms are diminishing enough to deem future convergence, provided it diminishes until below the threshold. Note that it is deemed, which means it could be wrong. The C47/R47 has no crystal ball. The machine bets that the unseen tail of the function keeps diminishing, enough that it could not add the difference. That is a lot of words saying it guesses the tail will die out. It remains the user's choice whether to use the normal Σ_n or the $\infty\Sigma_n$ version.

Table 4 Six runs of $\infty\Sigma_n$, each asked for ten thousand iterations. Every answer is identical to the one Σ_n gives for the same run.

	SERIES	SDIGS	ITERATIONS	ANSWER
p.	$1/(n-1)!$, giving e	34	50	2.718281828459045235360287471352663
q.	$n^2/2^n$	34	138	6
r.	$1/n^2$	34	10000	1.644834071848059769806081833310311
s.	$1/n^2$	7	9984	1.644834
t.	$1/n$	7	10000	9.787606
u.	$n^2/1.1^n$	34	10000	2310

The factorial series (p) completes at the fiftieth iteration, the earliest it may stop, and $n^2/2^n$ (q) at the hundred and thirty eighth iteration. Both (p) and (q) grow a bit before they fall, and $n^2/2^n$ (q) peaks at its third term, which is before the watch point being 10.

The four rows (r) to (u) show the rule declining to stop, each for its own reason. At 34 digits the $1/n^2$ series (r, s) never satisfies Formula 2: its terms fall as $1/m^2$ while the iterations left fall only as $N - m$, so the product stays above the last digit of the total until the count runs out. At seven digits (s) the same product crosses the line at $m = 9984$ and the last sixteen iterations are dropped. The harmonic series (t) fails that test more heavily still, its terms falling only as $1/m$. And $n^2/1.1^n$ (u) fails Formula 1 instead: it climbs until about its twenty first iteration, which is after the watch point, so the run is disqualified and counts to the end even though it decays steadily from there.

4 A typical program

Every sum in Table 1 was made with the program below, which is in `programs/sums.p47`. It is the sum of that table: $1/n^2$ for $n = 1$ to 10000.

FSQ is the term. The engine calls it once per count with n in **X**, and it leaves $1/n^2$ there for the engine to collect. EUP is the run: the three numbers, then the engine, naming FSQ as the term to call.

```

LBL 'FSQ'
  1/x
  x^2
RTN

LBL 'EUP'
  1
  10000
  1
  Σn 'FSQ'
RTN

```

From the keyboard it is the same three numbers, then Σ_n on the second soft key of **f** **ADV**, then the

program name at the prompt. Read the line as: from 1, to 10000, in steps of 1, do the sum.

1 ENTER 10000 ENTER 1 f ADV Σ FSQ

Counting down is the same three numbers the other way about, with the step made negative:

10000 ENTER 1 ENTER 1 CHS f ADV Σ FSQ

The two + lines of Table 1 came from the same program with the end of FSQ changed. Instead of leaving the term for the engine, it adds the term to a variable of its own and hands the engine a zero, so the running total is SS and the engine's 75 digits are never used:

```
LBL 'CSQ'
  1/x
  x2
  RCL 'SS'
  +
  STO 'SS'
  0
  RTN
```

The CSQ term program is a hack, using the looping of Σ_n but a register for every addition. It is not the usual way to write a sum. It is here to illustrate the loss of digits in a 34 digit run.

Clear SS before the run and read it afterwards. This is the version whose counting direction matters.

5 In summary

- Let Σ_n do the adding using a programmed sum, not a manual ST0+ or similar.
- Σ_n never stops early. Positive and negative count directions are both possible, both with fixed terms.
- $\infty\Sigma_n$ stops as soon as it is deemed the last digit cannot change any more, and counts up or down as Σ_n above.

Colophon

Body and headings	TeX Gyre Heros, the metric-compatible Helvetica from TeX Live
Mathematics	New Computer Modern Sans Math, from TeX Live
Display face	JuliaMono, from juliaweb.org
Typeset with	lua _l atex, TeX Live 2025
Design language	c47appnote v1.1, docs/appnotes/sources/style
Charts	pgfplots, one hue and the grey magnitude ramp
Screen captures	the GTK simulator at --reset, upscaled 3× with a point filter
Written against	firmware 00.109.04.00b0

AUTHOR **Jaco Mostert**

Copyright (C) 2026 The C47 Development Team. Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license can be downloaded from gnu.org.